

Reviewer Pack — Minimal Order of Symplectic Leaves and Communication Complex...

Entry 9b9adfab7928 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 02:51:40 UTC

Minimal Order of Symplectic Leaves and Communication Complexity Rank

Entry ID: 9b9adfab7928 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 02:51:40 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex symmetric Boolean function f , the minimal order of leaves in the symplectic Grassmannian associated with f is linearly correlated with its communication complexity rank $R(f)$, such that $O(n) = \Theta(R(f))$.

Rationale (proposer's reasoning):

Symplectic geometry has seen limited applications to computational complexity theory. The symplectic leaves represent a geometric structure that could potentially capture the essence of communication complexity, which deals with information exchange between parties. This conjecture proposes a quantitative relationship between these two concepts.

Taxonomy category: SymplecticGeometryToCommunicationComplexityRank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d212bfa1bc76ab0a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the linear regression correlation coefficient between the minimal order of leaves (MOL) and communication complexity rank ($R(f)$) exceeds 0.7 for all n -vertex symmetric Boolean functions f , across at least 30 seeds, with a p -value less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "communication complexity rank" AND minimal order" - "Grassmannian leaves" related to ("symplectic geometry" AND "matrix rank")" - "symplectic Grassmannian" intersection with ("communication complexity" AND "rank")

Top relevant hits considered: - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/2210.01601v2] Quantum communication complexity of linear regression - [http://arxiv.org/abs/2103.16512v2] The tropical symplectic Grassmannian

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=246.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_symmetric_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        # Placeholder function to calculate R(f)
        # Replace with actual algorithm or software package
        return len(f) / 2

    def symplectic_grassmannian(f):
        # Placeholder function to compute the symplectic Grassmannian
        # Replace with actual implementation using SymPy or other library
        return len(f) # Simplified for demonstration

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_symmetric_boolean_function(n)
        mol = symplectic_grassmannian(f)
        R_f = communication_complexity_rank(f)
        results.append((n, mol, R_f))
```

```

if len(results) < 30:
    return {
        "seed": seed,
        "metric_name": "communication_complexity_rank",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n for n, _, _ in results),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mol_values = [mol for _, mol, _ in results]
R_f_values = [R_f for _, _, R_f in results]

def mean(lst):
    return sum(lst) / len(lst)

def variance(lst, avg):
    return sum((x - avg) ** 2 for x in lst) / len(lst)

mol_avg = mean(mol_values)
R_f_avg = mean(R_f_values)
mol_var = variance(mol_values, mol_avg)
R_f_var = variance(R_f_values, R_f_avg)

covariance = sum((mol_values[i] - mol_avg) * (R_f_values[i] - R_f_avg) for i in range(len(results)))
correlation_coefficient = covariance / math.sqrt(mol_var * R_f_var)

return {
    "seed": seed,
    "metric_name": "communication_complexity_rank",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for n, _, _ in results),
    "conjecture_holds": abs(correlation_coefficient) >= 0.7,
    "counterexample": "" if abs(correlation_coefficient) >= 0.7 else "correlation_coefficient="
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
        support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])

```

```

print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient<0.7\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time constraints.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	21459
2	preregistration	ollama_remote	glm4:latest	0	0	14048
3	novelty	ollama_remote	glm4:latest	0	0	8744
4	novelty	ollama_remote	glm4:latest	0	0	10173
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16156
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47828
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33212
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11686
9	judge	ollama_remote	glm4:latest	0	0	26989

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 190295 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9b9adfab7928.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/9b9adfab7928.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9b9adfab7928.tar.gz` (if generated)